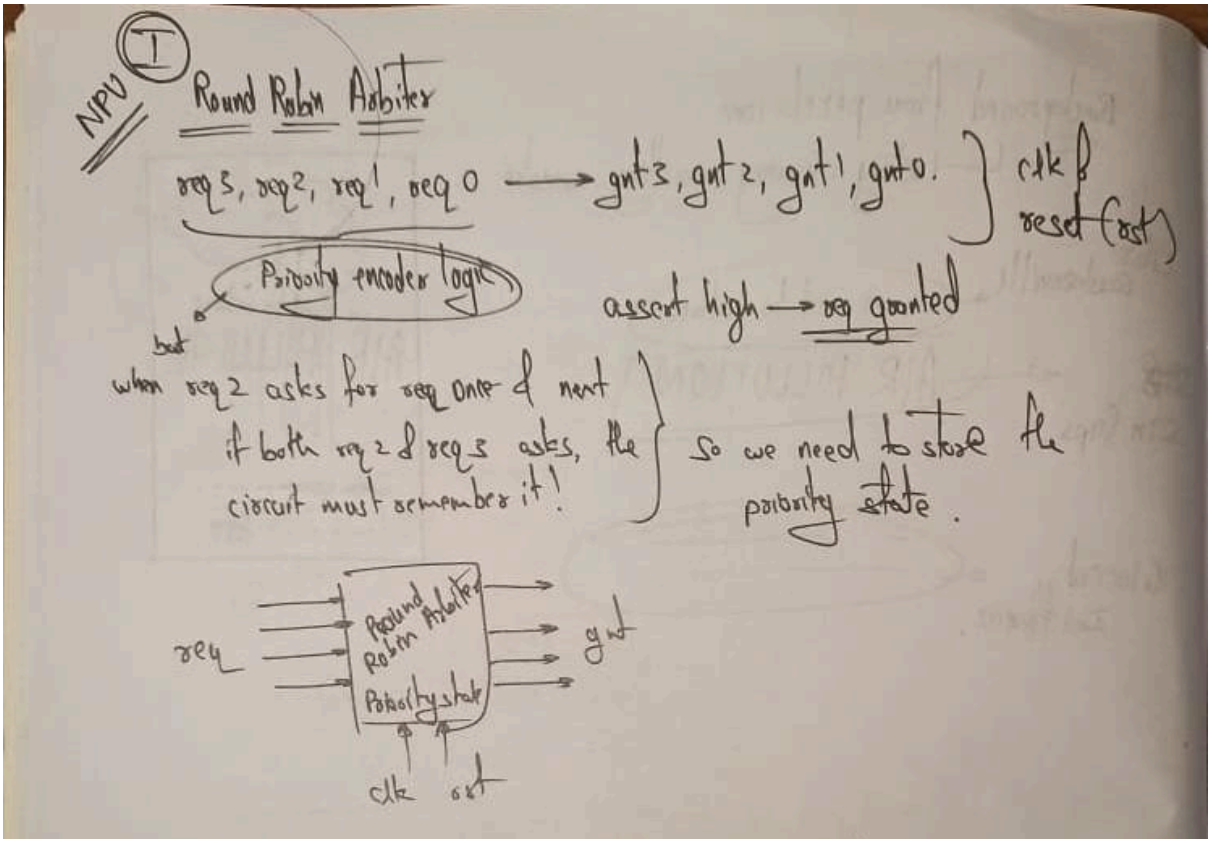
RISC V Based NPU

Assignment

Problem statement 1

Design a 4-request Round Robin Arbiter that grants access to one requester at a time in a fair and cyclic manner.

My Idea and Understanding of the question.



Please note that the explanation of code and justification of RTL logic has been done in my codes themself through comments as well as my own handwritten understandings. That's the reason i havent written anything else in this word document. This is just for a clean walk through my works. All the verilog codes and output waveform pngs have been included in the files very systematically. Thank you :)

Verilog code:

roundrobinarbiter.v* | tb_roundrobinarbiter.v

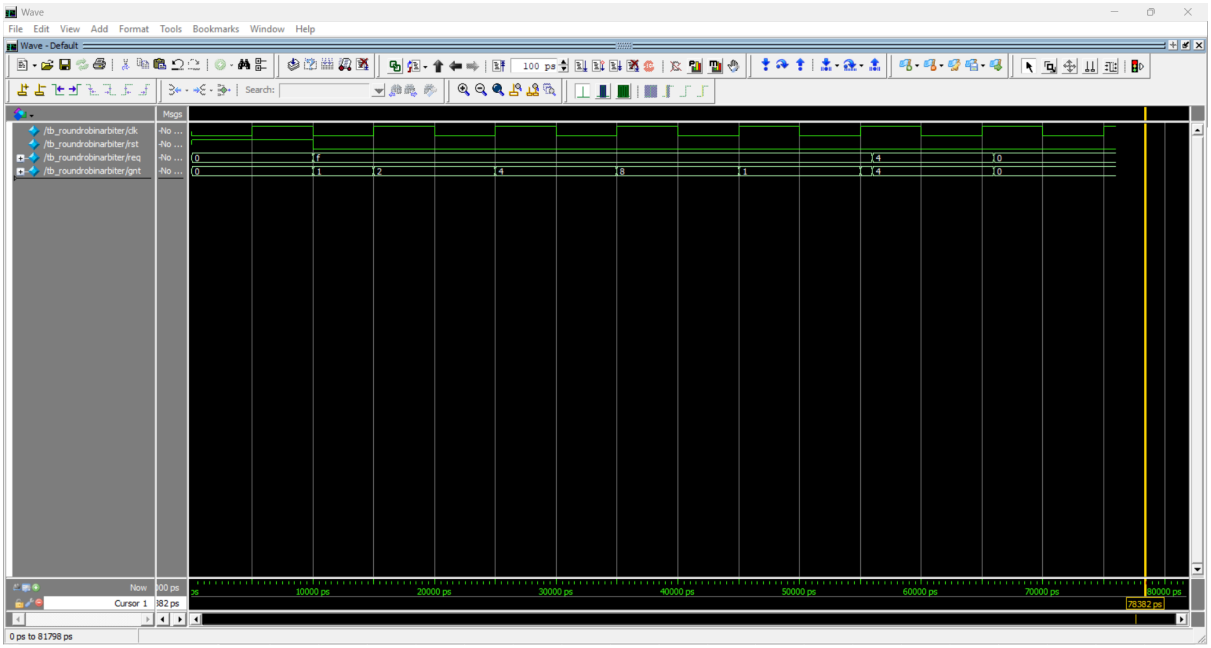
```
1  module roundrobinarbiter(
2      input wire clk,
3      input wire rst,
4      input wire [3:0]req,
5      output reg [3:0]gnt);
6
7  reg [1:0] priority; /*The arbiter needs to remember which requester should get the priority next
8  as there are four bit requests, we will assign four values to the priority register aswell
9  two bits [1:0] can represent four values/states (0,1,2,3)*/
10
11
12 always @(posedge clk or posedge rst) begin
13     if (rst) //whenever reset is high, the priority rresets to zero state
14         priority <= 2'd0;
15     else if (|gnt) begin //|gnt checks whether any bit in gnt is 1(hihg)
16                          //If not then the block doesnt execute and no request is granted
17         if (gnt[0])
18             priority <= 2'd1;
19         else if (gnt[1])
20             priority <= 2'd2;
21         else if (gnt[2])
22             priority <= 2'd3;
23         else
24             priority <= 2'd0; //priority is given to whatever bit of gnt is high
25     end
26     //update priority after each gnt
27 end
28
29 reg found; //checks if any requester has been selected or not
30           //because only one requester should recieve the grant
31 integer i;
32 integer idx; //for storing inder of currrent requester
33
34
35 always @(*) begin //combonational block
36     gnt = 4'b0000;
37     found = 1'b0;
38     idx = 0;
39
40     for (i = 0; i < 4; i = i + 1) begin
41         idx = (priority + i) % 4; /*to ensure that whenever priority state is assigned
42         it should always reset to zero once the cycle is completed
43         basically 0-> 1-> 2-> 3-> 0 -> 1......*/
44
45         if (!found && req[idx]) begin /*if found = 0, !found = 1, hence we ensure one condition is satisfied
46         which was to maske sure no active requests are there when assigning a new request*/
47
48         /*also, req[idx] selects that the current requester only is requesting access*/
49             gnt[idx] = 1'b1; //thr request is granted to the idx'th requester
50             found = 1'b1; // found is set to 1 as a requester has been selected now
51         endn
52     end  //checks requests in roundrobin order
53 end
54
55 endmodule
```

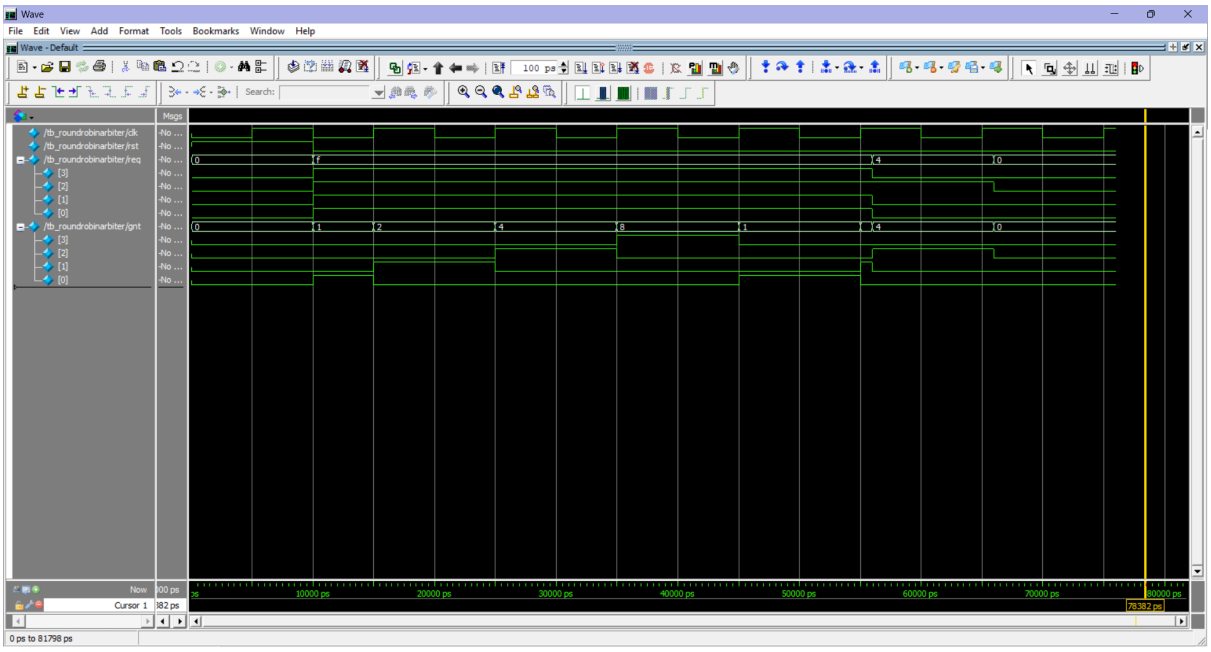
Testbench code:

roundrobinarbiter.v* | tb_roundrobinarbiter.v

```
1   `timescale 1ns/1ps
2
3   module tb_roundrobinarbiter;
4
5   reg clk;
6   reg rst;
7   reg [3:0]req;
8   wire [3:0]gnt;
9
10
11  roundrobinarbiter uut ( //instantiation
12      .clk(clk), .rst(rst), .req(req), .gnt(gnt)
13  );
14
15  always #5 clk = ~clk;
16  initial begin
17      clk = 0;
18      rst = 1; //initially resetting for req[0] for 10ns
19      req = 4'b0000;
20          #10; rst = 0;
21
22           req = 4'b1111; //a case where all requesters are requesting access
23
24
25          repeat (4) begin
26              @(posedge clk);
27              #1; //obseriving all four arbitration cycles
28              $display("Time=%0t req=%b gnt=%b priority=%d",
29                       $time, req, gnt, uut.priority);
30          end
31
32          @(posedge clk);
33          #1; //checking if the arbiter starts another round
34          $display("Time=%0t req=%b gnt=%b priority=%d",
35                   $time, req, gnt, uut.priority);
36
37
38          req = 4'b0100;
39          @(posedge clk);
40          #1; //testing a single requester
41          $display("Single request: req=%b gnt=%b", req, gnt);
42
43
44          req = 4'b0000;
45          #10; //testing with no requests
46          $display("No requests: req=%b gnt=%b", req, gnt);
47
48          $finish;
49      end
50
51  endmodule
```

Waveforms:

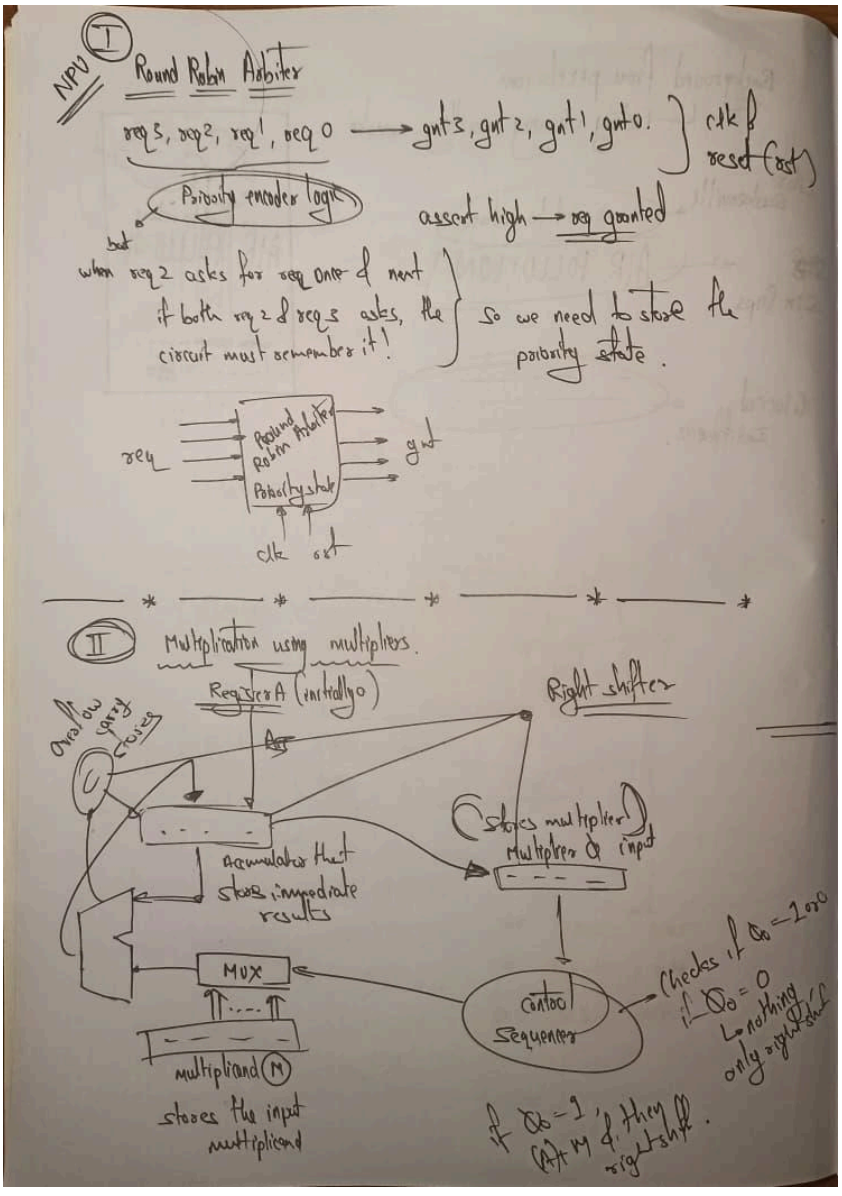

Wave
File Edit View Add Format Tools Bookmarks Window Help
Wave - Default
Msgs
/tb_roundrobinarbiter/clk
/tb_roundrobinarbiter/rst
/tb_roundrobinarbiter/req
/tb_roundrobinarbiter/gnt
Now
Cursor 1
10000 ps
20000 ps
30000 ps
40000 ps
50000 ps
60000 ps
70000 ps
80000 ps
78382 ps
0 ps to 81798 ps


Wave
File Edit View Add Format Tools Bookmarks Window Help
Wave - Default
Msgs
/tb_roundrobinarbiter/clk
/tb_roundrobinarbiter/rst
/tb_roundrobinarbiter/req
/tb_roundrobinarbiter/gnt
Now
Cursor 1
10000 ps
20000 ps
30000 ps
40000 ps
50000 ps
60000 ps
70000 ps
80000 ps
78382 ps
0 ps to 81798 ps

Problem statement 2

● Write the Verilog Code to model the Optimized Multiplier (n = 4). ● Develop a Testbench to demonstrate multiplication of 2, 4 bit unsigned numbers. ● Verify the Design's Correctness via Waveform Simulation.

My Idea and Understanding of the question, an explanation of optimised multiplier in my own words. Also stepwise computation (Using Pen and Paper) of the given multiplications examples showing how it would be performed by the Optimized Multiplier



Example M = 1101 (13) & Q = 1011 (11)

I st cycle Add → A = 1101 & C = 0, Q = 1011
I st cycle shift → A = 0110, Q = 1101, C = 0

II nd cycle Add → A+M => A = 0011, C = 1, Q = 1101
II nd " shift → A = 1001, Q = 1110, C = 0.

III rd cycle no add ($Q_0 = 0$) → A = 1001, Q = 1110, C = 0
III rd cycle shift → A = 0100, Q = 1111

IV th cycle add → A+M => A = 0001, C = 1, Q = 1111
IV th cycle shift → A = 1000, Q = 1111

Answer = $(10001111)_b = (143)_d$

1101
1011
1101
0000
1101
10001111

simple binary multiplication

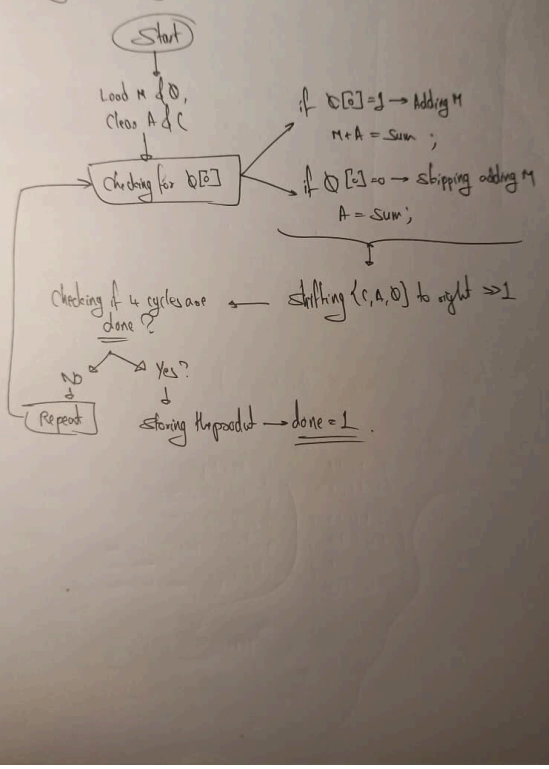
using a same 4 bit adders in four cycles for 2-4 bit number multiplication
It's following the same logic of the normal binary multiplication

1101
1011
① 1101
② 1101 +
③ 0000 + c
④ 1101 + + +
10001111

after every multiplication step we right shift & then add

10001111 (right shifting)

Verilog Code working



Step wise computation of the following examples.

Q1) M = 1011 (11) & Q = (0111)(7)

	S	A	Q	checking	cycle
	0	1011	0111	$Q_0 = 1$	I (Add) ✓
	0	0101	1011		I (shift)
	1	0000	1011	$Q_0 = 1$	II (Add) ✓
	0	1000	0101		II (shift)
	1	0011	0101	$Q_0 = 1$	III (Add) ✓
	0	1001	1010		III (shift)
	0	1001	1000	$Q_0 = 0$	IV (Add) X
	0	0100	1000		IV (shift)

$(01001101)_b = (77)_d$ => 11 x 7 = 77

Q2) M = 0110 (6) & Q = 1001 (9)

C	A	Q	Check	cycle
0	0110	1001	$Q_0 = 1$	Add (I)
0	0011	0100		shift (I)
0	0001	1010	$Q_0 = 0$	~~Add~~ shift (II)
0	0000	1101	$Q_0 = 0$	~~Add~~ shift (III)
0	0110	1101	$Q_0 = 1$	Add (IV)
0	0011	0110		shift (IV)

$(00110110)_b = (54)_d$ => 6 x 9 = 54

Verilog code:

optimizedmultiplier.v | tb_optimizedmultiplier.v

```
1  module optimizedmultiplier(
2  input clk, rst, srt,
3  input [3:0]M, Qin,
4  output reg [7:0]prod,
5  output reg done);
6
7  reg [3:0] A, Q, Mreg; //A = partial result of sum of M + A, Q = stores multiplieer, Mreg to store the M
8  reg C; //stores carry (single bit)
9  reg [2:0] count; // counts cycles, three bits (7 stages) counts 4 addition stagesa nd 4 shift stages
10 reg busy; //to prevent new operation form iteratiion when a multiplication is already goin'on
11 reg [4:0] sum;
12 reg [8:0] temp;
13
14
15 always @(posedge clk or posedge rst) begin
16     if (rst) begin //reseting everything
17         A <= 0; Q <= 0; Mreg<= 0; C <= 0; count <= 0; prod <= 0; done <= 0; busy <= 0;
18     end
19     else begin
20             done <= 0; //should stay low while multiplication is running, high only when completed
21
22         if (srt && !busy) begin //process begins when multiplication has been requested and the system aint busy
23                 A <= 0; //initial state
24                 Q <= Qin;
25                 Mreg <= M; //register has been valued with M
26                 C <= 0; //carry is resetetd
27                 count <= 0; //the count starts from now
28                 busy <= 1; //system is activated
29         end
30
31         else if (busy) begin //now when ths system is busy, i.e when active
32
33             if (Q[0]) //checking the multipliers(Q's) rightmost bit (Q0)
34                 sum = {1'b0, A} + {1'b0, Mreg}; // here the M which was already assigned is being used to add with current A
35             else
36                 sum = {1'b0, A}; // if not then A stays as A/ no addition of M
37
38             temp = {sum, Q} >> 1; /* concatinating the sum and Q to right shift thenm together by a, note that we are
39             operating in an uunsigned system, the gap of leftmost bit is filled by 0, when the right shoting is
40             completed, its then stored in C, A and Q respectively to give them new values*/
41
42             //we concatinated sum and Q as our product will also be in that form/ and its just easier to deal with
43             C <= temp[8]; A <= temp[7:4]; Q <= temp[3:0];
44
45             if (count == 3) begin /*iteration 0 is stage one so when count is 3, i.e stage is four, the process comes
46             to an halt. the values are assigned to the final outcomes and its done*/
47                 prod <= temp[7:0];
48                 busy    <= 0;
49                 done    <= 1;
50             end
51
52             count <= count + 1'b1; //system doesnt proceed/ gives output after 3rd count.
53         end
54     end
55 end
56
57 endmodule
```

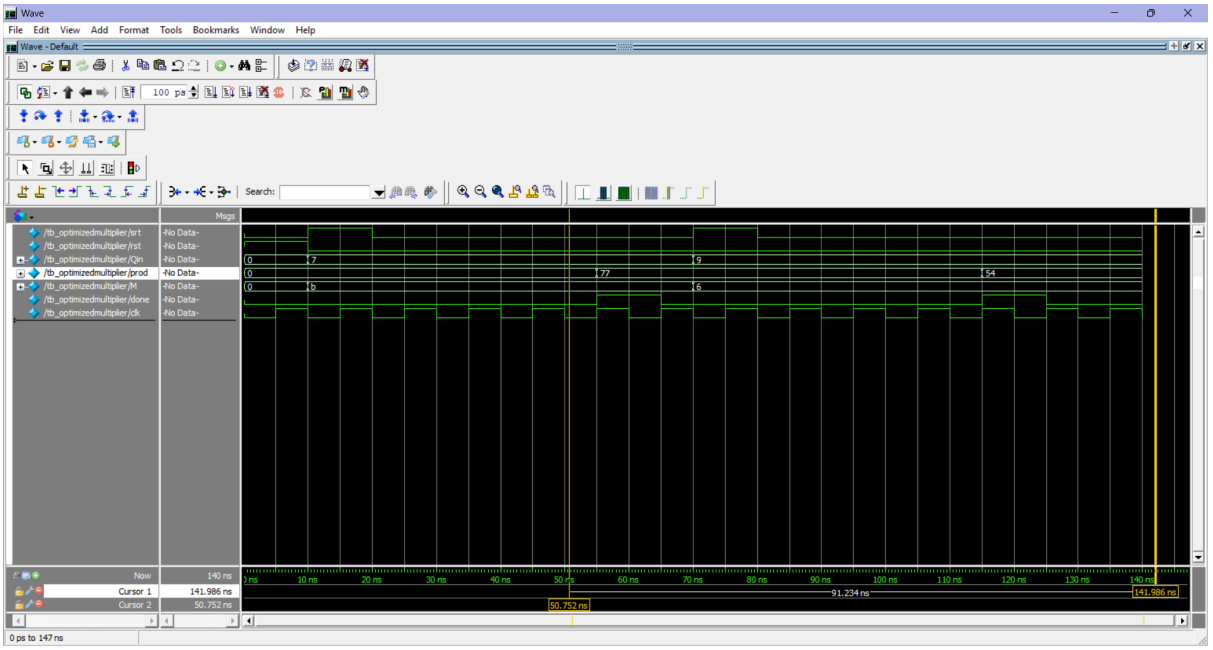
Testbench code:

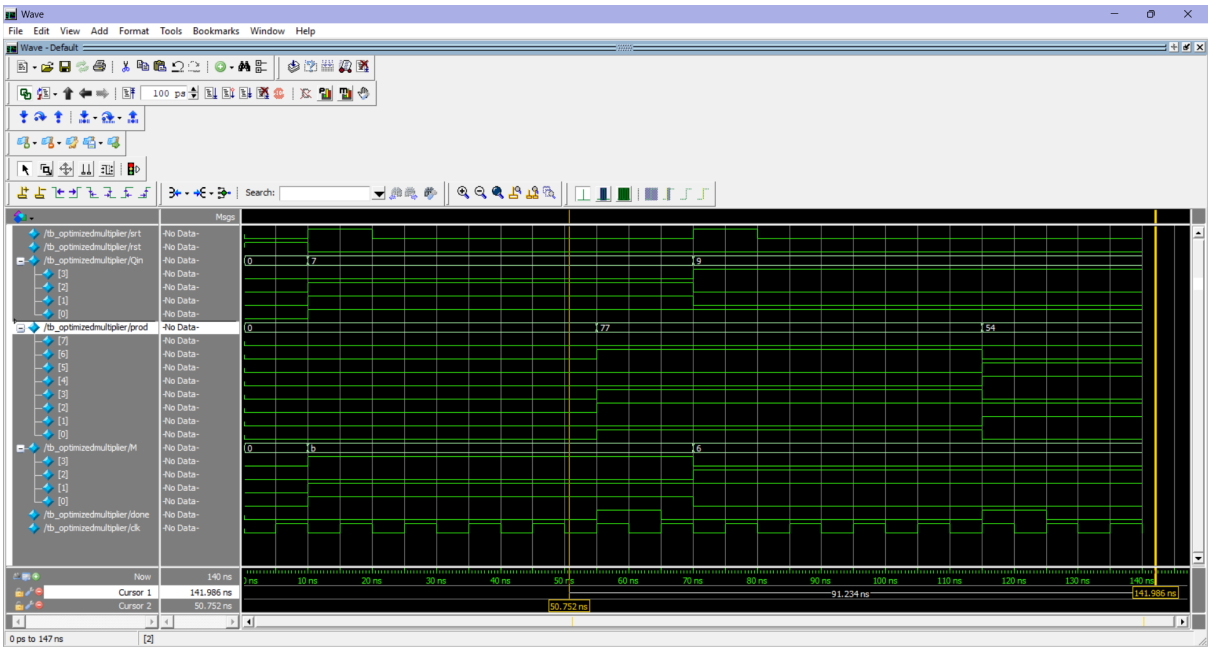
optimizedmultiplier.v | tb_optimizedmultiplier.v

D:/ACM/NPU/Quartus/Problem statement 2/tb_optimizedmultiplier.v

```
1  `timescale 1ns/1ps
2
3  module tb_optimizedmultiplier;
4  reg clk;
5  reg rst;
6  reg srt;
7  reg [3:0] M;
8  reg [3:0] Qin;
9
10 wire [7:0] prod;
11 wire done;
12
13 optimizedmultiplier uut (//instantiation
14         .clk(clk), .rst(rst), .srt(srt), .M(M), .Qin(Qin), .prod(prod), .done(done)
15     );
16
17 always #5 clk = ~clk;
18 initial begin
19
20 clk = 0; rst = 1; srt = 0; M = 0; Qin  = 0;
21 #10; rst = 0;
22
23 //11*7=77 case
24 @(negedge clk);
25 M = 4'b1011;
26 Qin = 4'b0111;
27 srt = 1;
28
29 @(negedge clk);
30 srt = 0;
31 wait(done == 1); //multiplication completed
32 repeat(2) @(negedge clk);
33
34 //6*9=54 case
35 M = 4'b0110;
36 Qin = 4'b1001;
37 srt = 1;
38
39 @(negedge clk);
40 srt = 0;
41 wait(done == 1);
42 repeat(3) @(negedge clk);
43
44 $stop;
45 end
46
47 endmodule
```

Waveforms:





Thankyou :)